

Lecture 2

Assessing Model Accuracy

2.1

Contents

1 Assessing Model Accuracy	1
2 Measuring the Quality of Fit	1
2.1 The <i>training MSE</i> and the <i>test MSE</i>	2
2.2 Application	3
3 The Bias-Variance Trade-Off	9

Some figures in this presentation are taken from 'An Introduction to Statistical Learning, with applications in R' (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani.

Most of the material is extracted from the slides of the MOOC course by Hastie and Tibshirani, available at <https://www.r-bloggers.com/in-depth-introduction-to-machine-learning-in-15-hours-of-expert-videos/>.

2.2

1 Assessing Model Accuracy

Goal

Select the best statistical learning approach to analyze a specific data set

2.3

2 Measuring the Quality of Fit

We will consider a generic statistical learning method aimed at predicting an outcome Y as a function of a predictor X by means of the estimate $\hat{f}$ of f in the relationship¹

$$Y = f(X) + \varepsilon,$$

where f is some fixed but unknown function of X and ε is a random error term.

The available training data set $\{(x_1, y_1), \dots, (x_n, y_n)\}$ contains information about variables X and Y .

In order to evaluate the performance of a statistical learning method the most commonly used measure is the mean squared residual, or *Mean Square Error (MSE)*, computed on the training data set

$$MSE_{\text{training}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \quad (1)$$

¹ $f(x) = a + bx$ with bivariate linear regression model and

$$Y = a + bX + \varepsilon.$$

where:

- y_i is the value of Y for the i th observation,
- $\hat{f}(x_i)$ is the prediction that $\hat{f}$ gives for the i th observation.

2.4

2.1 The *training MSE* and the *test MSE*

The *training MSE* is computed on the data used to estimate $\hat{f}$.

We are interested in evaluating the predictive ability of the statistical learning method on new data.

Consider for example:

- the forecasts of future values of a stock or of a macroeconomic indicator;
- the assessment by a bank of the credit quality of its borrowers;
- the profiling of new customers of an insurance company.

Therefore, we need to have recourse to the so-called *test MSE*.

2.5

The training set

$$\{(x_1, y_1), \dots, (x_n, y_n)\}$$

is used only for obtaining the estimate $\hat{f}$ of f . The *test MSE* is computed on *new* data

$$\{(x_{01}, y_{01}), \dots, (x_{0k}, y_{0k})\}$$

(that were not used to estimate $\hat{f}$).

We have

$$MSE_{test} = \frac{1}{k} \sum_{i=1}^k (y_{0i} - \hat{f}(x_{0i}))^2. \quad (2)$$

2.6

Evaluation with the *training MSE*

$$\begin{array}{ccccc} \text{training data set} & & \text{estimate of } f & & \text{training MSE} \\ & & & & \text{comparison between} \\ \{(x_1, y_1), \dots, (x_n, y_n)\} & \rightarrow & \hat{f} & \rightarrow & \hat{f}(x_1) \quad \hat{f}(x_2) \quad \dots \quad \hat{f}(x_n) \\ & & & & \text{and} \\ & & & & y_1 \quad y_2 \quad \dots \quad y_n \end{array}$$

2.7

Evaluation with the *test MSE*

$$\begin{array}{ccc} \text{training data set} & & \text{estimate of } f \\ \{(x_1, y_1), \dots, (x_n, y_n)\} & \rightarrow & \hat{f} \\ \\ \text{test data set} & & \text{test MSE} \\ & & \text{comparison between} \\ \{(x_{01}, y_{01}), \dots, (x_{0k}, y_{0k})\} & \rightarrow & \hat{f}(x_{01}) \quad \hat{f}(x_{02}) \quad \dots \quad \hat{f}(x_{0k}) \\ & & \text{and} \\ & & y_{01} \quad y_{02} \quad \dots \quad y_{0k} \end{array}$$

2.8

How can we proceed?

- If a test data set is available
→ compute the MSE_{test} by means of (2) and choose the statistical learning method with the minimum MSE_{test} .

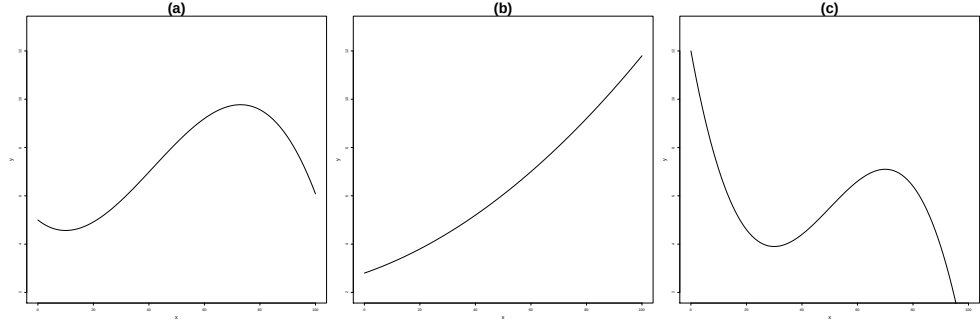


Figure 1: theoretical relationships used to simulate data

- If a test data set is not available
 → it is only possible to compute the $MSE_{training}$ according to (1)
 there is no guarantee that the method with the lowest $MSE_{training}$ will also have the lowest MSE_{test} .

2.9

Usually the following relationship holds

$$MSE_{test} > MSE_{training}.$$

For example, the OLS estimate of the parameters in the linear regression model

$$Y = \beta_0 + \beta_1 X + \varepsilon.$$

is obtained by minimizing the MSE on the training data set

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{n} \sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_i)]^2 \right\} = \arg \min_{\beta} MSE_{training}.$$

2.10

2.2 Application

Example 1. We assume to know the theoretical relationships linking the outcome Y to the predictor X in three different data sets.

We compute the $MSE_{training}$ and MSE_{test} corresponding to the estimates:

- of a linear model $\hat{f} = a + bx$;
- of the *smoothing spline* model that has the minimum MSE_{test} (in this example this will result a model slightly more flexible than the linear model);
- of a *smoothing spline* with 35 degrees of freedom (a highly flexible model).

2.11

In order to have an idea of what is a smoothing spline model, one can have a look at Figures 2.4-2.6 in JWHT.

2.12

Figure 1 plots the theoretical relationships used to simulate the three data sets. In the

2.13

second example the relationship is close to linear. For the first and third data sets the

2.14

relationships are not so close to linear; the relationship is far from linear in situation (c).

2.15

A noise component is present in the data generating model. We assume that it follows a standard Normal distribution and that errors are i.i.d. between statistical units

2.16

2.17

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim N(0, 1).$$

Remind in the following discussion that the variance of the noise has been set equal to $\sigma_E^2 = 1$.

We perform model evaluation on a test data set of size $n = 10000$.

2.18

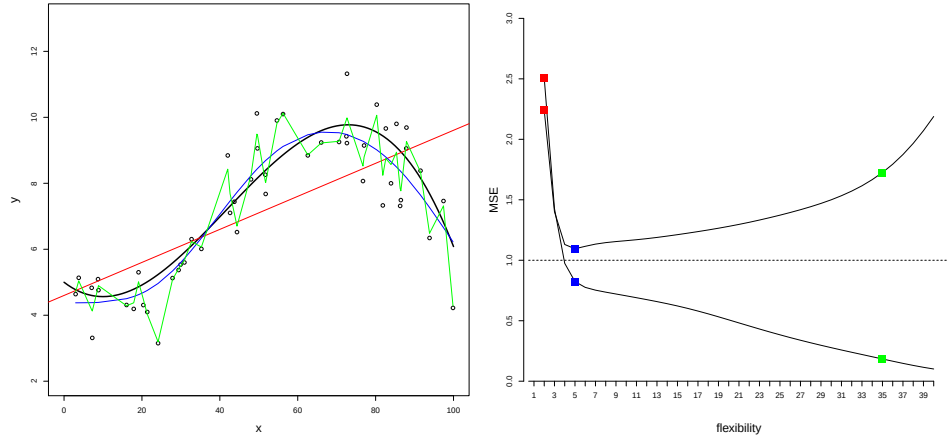


Figure 2: The theoretical relationship $f(x) = 5 - 41.667\left(\frac{x}{100}\right)^3 + 51.875\left(\frac{x}{100}\right)^2 - 9.125\left(\frac{x}{100}\right)$ is non linear. The smoothing spline model with 5 degrees of freedom (blue line) fits quite good the theoretical relationship f . The model with 35 dof (green line) is less smooth; it fits better observed values, but has a lower level of predictive ability (overfitting problem).

The graph on the right side shows that the $MSE_{training}$ decreases when the flexibility of the model increases; the MSE_{test} initially decreases (compare values for the model with 2 dof – that corresponds to linear – and the model with 5 dof) and then increases when increasing the level of flexibility, showing overfitting problems.

Figures 2, 3 and 4 represent:

- left plot
 - the theoretical relationship $f(x)$ (bold line);
 - the simulated data (training data set);
 - the estimate of the theoretical relationship according to a linear model (red line), and the two smoothing spline models (the blue line is the model with lowest MSE_{test} and the green line the model with 35 degrees of freedom);
- the plot on the right side shows the behaviour of the $MSE_{training}$ and MSE_{test} when the flexibility of the smoothing spline model increases (flexibility is measured by using the number of degrees of freedom). The smoothing spline model with 2 degrees of freedom corresponds to linear.

Comments for the 3 data sets are in the figure captions.

Observe that the MSE_{test} was evaluated on a test data set of 10000 statistical units by having recourse to the estimate $\hat{f}$ computed on the training set of 50 statistical units that are represented in the left side of the figure.

Therefore, the behaviour of MSE_{test} as well as the accuracy of the considered models depend on the training set that was used for estimation.

In order to estimate the theoretical behaviour of MSE_{test} we will consider an estimate of MSE_{test} that is obtained by varying also the training set, see Figure 5.

2.19

2.20

2.21

2.22

2.23

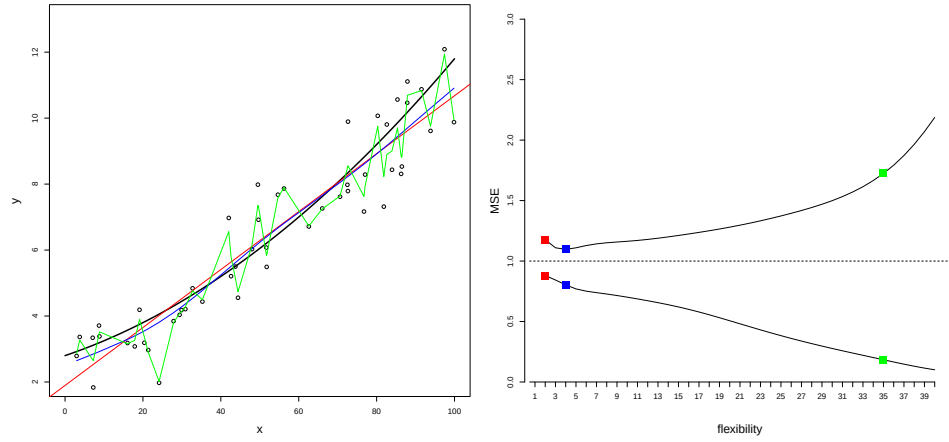


Figure 3: The theoretical relationship $f(x) = 2 + 5 \left(\frac{x}{100}\right)^2$ is very close to linear. Both the linear and the smoothing spline (4 dof) models (blue line) fit quite close the theoretical relationship f . The model with 35 dof (green line) is very wiggly; it fits data more closely, but presents a lower level of predictive ability (overfitting).

The graph on the right side shows that the $MSE_{training}$ decreases when the flexibility of the model increases; the MSE_{test} has a small initial reduction, from 2 to 4 dof, since the theoretical model is very close to linear, then it increases with increasing flexibility, showing overfitting problems.

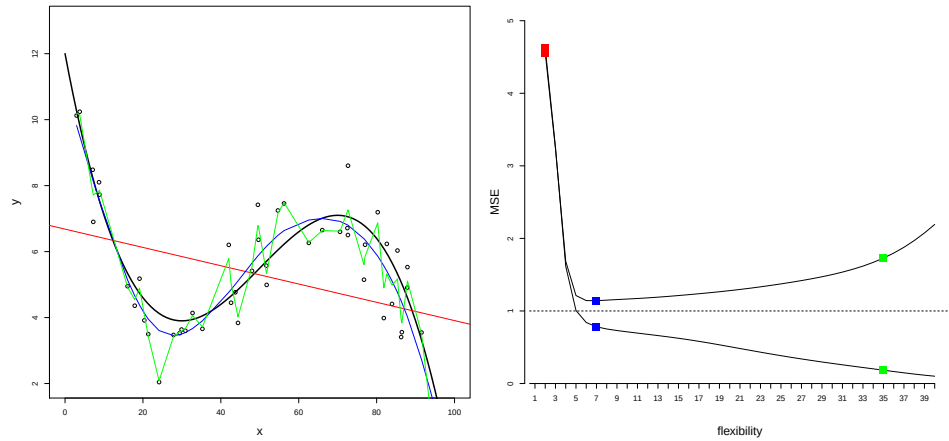


Figure 4: The theoretical relationship $f(x) = 12 - 100 \left(\frac{x}{100}\right)^3 + 150 \left(\frac{x}{100}\right)^2 - 63 \left(\frac{x}{100}\right)$ is very far from linear (also with respect to the first data set). The smoothing spline model with 7 degrees of freedom (blue line) fits quite closely the theoretical behaviour of f . The model with 35 dof (green line) has a more wiggly behaviour; it fits data more closely, but has a lower predictive ability (overfitting problem).

The graph on the right side shows that the $MSE_{training}$ decreases when the model flexibility increases, while the MSE_{test} has a rapid initial decrease (compare values for the model with 2 dof – that corresponds to linear – and the model with 7 dof) and then increases when increasing the level of flexibility, showing overfitting problems.

Observe the 'special' features of the 3 considered data sets. The theoretical relationship between the outcome and the explanatory variable was known, and it was possible to generate a test data set, different from the training data set, on which evaluate the different models by having recourse to the MSE_{test} .

The $MSE_{training}$ decreases when the flexibility level of the statistical learning method increases. Too much flexible models can cause overfitting situations, with $MSE_{training}$ lower than 1 (the variance of the noise in the data generating model) and high values of the MSE_{test} . These models follow too much closely the data, in the sense that they follow the errors, or noise, instead of the structure f .

We will describe how to find the optimal level of flexibility of a statistical learning method when a test data set is not available. We will have recourse to the so-called cross validation procedures.

2.24

We report the code to reproduce the figures

- generation of the training set ($n = 50$ statistical units);
- estimate the parameters of a linear model $\hat{f} = \hat{a} + \hat{b}x$;
- estimate $\hat{f}$ by means of a smoothing spline model with minimum MSE_{test} ;
- estimate $\hat{f}$ by means of a smoothing spline model with 35 degrees of freedom (highly flexible model);
- plot of the $n = 50$ pairs (x_i, y_i) and of their fit with the 3 models
- generation of a test data set ($n = 10000$ statistical units)
- plot of the smoothing spline $MSE_{training}$, MSE_{test} depending on different degrees of freedom²

2.25

```
# theoretical relationship
f <- function(x) 5 - 41.667 * (x/100)^3 + 51.875 * (x/100)^2 -
  9.125 * (x/100)
# layout produces multiple graphs on the same device
layout(matrix(1:2, 1, 2))
# plot theoretical relationship
curve(f, xlim = c(0, 100), ylim = c(2, 13), lwd = 3, xlab = "x",
  ylab = "y", cex.lab = 1.5)
```

2.26

```
# data simulation
set.seed(1234567) #set the random number seed (for reproducibility)
x <- runif(50) * 100
x <- sort(x)
y <- f(x) + rnorm(50)
# add scatter plot
points(x, y)
# OLS estimate of the linear model
lmest <- lm(y ~ x)
# OLS output
summary(lmest)
# coefficients
coef(lmest)
# add the line on the plot
abline(lmest, col = "red")
```

2.27

```
# model matrix
X <- model.matrix(lmest)
# alternative methods to obtain values fitted by the linear
```

²The linear model corresponds to a smoothing spline with 2 degrees of freedom.

```

# model
fitted1 <- X %*% coef(lmest)
fitted2 <- apply(t(X) * coef(lmest), 2, sum)
fitted <- lmest$fitted
# alternative methods to obtain residuals of the linear model
residuals1 <- y - lmest$fitted
residuals <- lmest$res
# training MSE
MSE.training.lm <- mean(lmest$res^2)

```

2.28

```

# simulation of the test data set
new.length <- 10000
x.new <- runif(new.length) * 100
# theoretical values according to relationship: f+noise
y.new <- f(x.new) + rnorm(new.length)
# linear model predicted values
y.new.pred <- predict(lmest, data.frame(x = x.new))
# test MSE
MSE.test.lm <- mean((y.new - y.new.pred)^2)

```

2.29

```

# function to compute training and test MSE for smoothing
# splines with varying degrees of freedom (dof)

# objects x and y contain data (they are in the general
# environment and are free variables for sm.funct)
sm.funct <- function(dof) {
  fit <- smooth.spline(x, y, df = dof)
  pred <- predict(fit, x)
  MSE.training <- mean((y - pred$y)^2)
  pred.new <- predict(fit, x.new)
  MSE.test <- mean((y.new - pred.new$y)^2)
  return(c(MSE.training, MSE.test))
}
# obtain training and test MSE for 2:40 dof
evaluation <- sapply(2:40, sm.funct)
# evaluation[1,] and evaluation[2,] contain the training and
# the test MSE values. which.min finds the number of degrees
# of freedom of the spline model with minimum MSE.test
df <- which.min(evaluation[2, ]) + 1

```

2.30

```

# estimate of the 'optimal' spline model
fit2 <- smooth.spline(x, y, df = df)
# predicted values for the smoothing spline model applied to
# the training data set
pred2 <- predict(fit2, x)
# plot the smoothing spline model
lines(pred2$x, pred2$y, col = "blue")
# training MSE of the smoothing spline model
MSE.training.2 <- mean((y - pred2$y)^2)
# predicted values of the smoothing spline model
pred2.new <- predict(fit2, x.new)
# test MSE of the smoothing spline model

```

```

MSE.test.2 <- mean((y.new - pred2.new$y)^2)
# second smoothing spline model
fit3 <- smooth.spline(x, y, df = 35)
pred3 <- predict(fit3, x)
lines(pred3$x, pred3$y, col = "green")
MSE.training.3 <- mean((y - pred3$y)^2)
pred3.new <- predict(fit3, x.new)
MSE.test.3 <- mean((y.new - pred3.new$y)^2)

```

2.31

```

# plot MSE.training and MSE.test on a new graph (plot is a
# high level graph command, it proceeds with the next layout
# value)
plot(2:40, evaluation[, ], type = "l", ylim = c(0, ceiling(max(evaluation))),
     xlab = "flexibility", ylab = "MSE", xaxt = "n", yaxt = "n",
     bty = "n", cex.lab = 1.5)
axis(1, pos = 0, cex = 2, at = 0:40)
axis(2, pos = 0, cex = 2)
lines(2:40, evaluation[, ])
# highlight the training and test MSE for the 3 estimated
# models
points(c(2, 2), evaluation[, 1], col = "red", pch = 15, cex = 2)
points(c(df, df), evaluation[, df - 1], col = "blue", pch = 15,
       cex = 2)
points(c(35, 35), evaluation[, 34], col = "green", pch = 15,
       cex = 2)
# irreducible variability level
abline(h = 1, lty = 2)

```

2.32

In order to reproduce results corresponding to the theoretical relationships shown in Figure 1 (b) and (c) (see Figure 3 and 4) it is sufficient to write a function, `ff`, whose main argument is the theoretical relationship `f`, that will be used to generate the data.

`ff` contains all the code that we used to produce Figure 2

```

ff <- function(f) {
  layout(matrix(1:2, 1, 2))
  curve(f, xlim = c(0, 100), ylim = c(2, 13), lwd = 3, xlab = "x",
        ylab = "y", cex.lab = 1.5)
  set.seed(1234567)
  ...
  ...
  ...
  abline(h = 1, lty = 2)
}

```

2.33

In order to generate data, we now define the theoretical functions `f2` and `f3`, and then apply function `ff`

```

f2 <- function(x) 2 + 5 * ((x + 40)/100)^2
ff(f2)

```

```

f3 <- function(x) -100 * (x/100)^3 + 150 * (x/100)^2 - 63 * (x/100) +
  12
ff(f3)

```

2.34

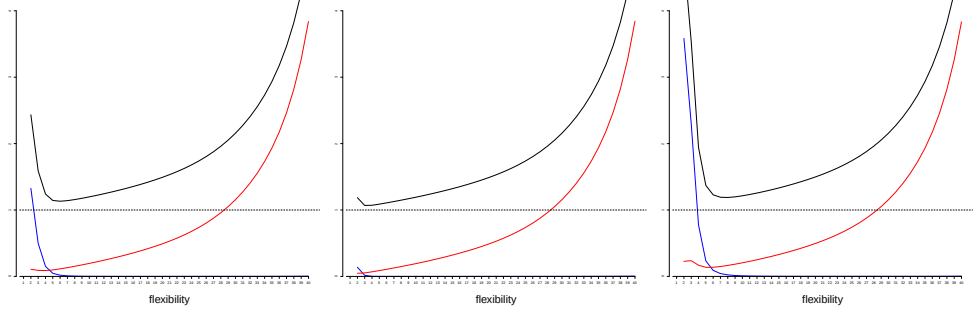


Figure 5: MSE_{test} behaviour (black bold curve) and its decomposition according to (3). $Var(\epsilon)$ horizontal line ($\sigma_\epsilon^2 = 1$); The square of bias $Bias_f^2$, blue curve, decreases when the flexibility increases; the variance of the estimation procedure of f , $Var(\hat{f}(x_0))$ first decreases and then increases when flexibility increases, giving rise to overfitted models.

3 The Bias-Variance Trade-Off

Decomposition of the MSE_{test}

In presence of a new observation x_0 we have

$$\begin{aligned}
 E \left[\{y_0 - \hat{f}(x_0)\}^2 \right] &= E \left[\{y_0 \pm f(x_0) \pm E(\hat{f}(x_0)) - \hat{f}(x_0)\}^2 \right] = \\
 &= E \left[\{[y_0 - f(x_0)] + [f(x_0) - E(\hat{f}(x_0))] - [\hat{f}(x_0) - E(\hat{f}(x_0))]\}^2 \right] = \\
 &= Var\{y_0 - f(x_0)\} + [f(x_0) - E(\hat{f}(x_0))]^2 + Var(\hat{f}(x_0)) = \\
 &= Var(\epsilon) + Bias_f^2 + Var(\hat{f}(x_0)) \quad (3)
 \end{aligned}$$

assuming that $Cov(\{y_0 - f(x_0)\}, \hat{f}(x_0)) = 0$.

$E \left[\{y_0 - \hat{f}(x_0)\}^2 \right]$ is to be interpreted as the expected value of the MSE_{test} (for x_0) computed by considering the estimates $\hat{f}(x_0)$ corresponding to a large number of training sets. The estimate of MSE_{test} is obtained by computing the average of the values of $MSE_{test}(x_0)$ on varying x_0 in the test set.

Figure 5 shows the decomposition of MSE_{test} . The estimate of MSE_{test} was obtained by averaging the $MSE_{test}(x_0)$ on varying x_0 in the test set and on varying the training data set.

We can observe

- the high variability in the estimate of $\hat{f}$ for the smoothing spline models when the number of degrees of freedom increases, that is when flexibility increases;
- the bias reduces when flexibility increases.

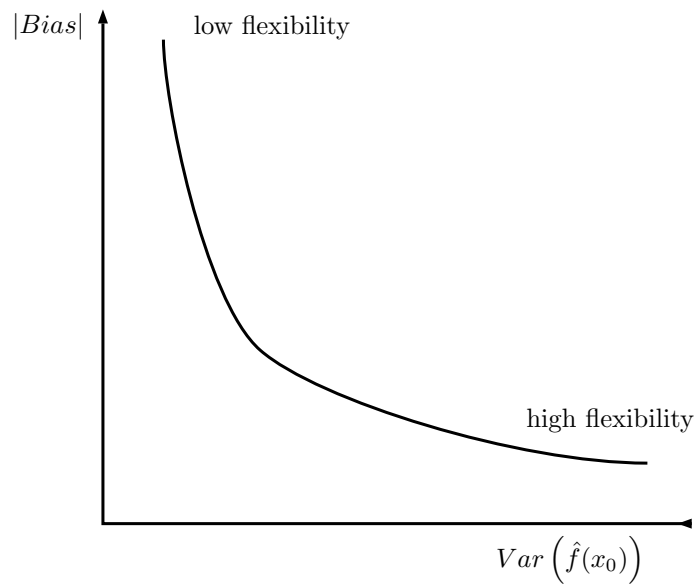
First-best solution

Find a statistical learning method that minimizes both $|Bias_{\hat{f}}|$ as well as $Var(\hat{f}(x_0))$.

- $Var(\hat{f}(x_0))$ measures the stability of the estimate $\hat{f}$ with different training sets. The variance is an increasing function of the level of flexibility of the statistical learning method. More flexible statistical learning methods are characterized by high variance, and result more sensible to variations in the data.

- $Bias_{\hat{f}}$ measures the error in approximating $f(x)$.
The bias is a decreasing function of the flexibility level of the statistical learning method.
For example, a linear approximation of a highly non-linear theoretical relationship f will have high error.

2.38



In the following section we will present some cross-validation techniques that allow the MSE_{test} to be estimated in presence only of the training data set.

2.39

Figure 5 can be reproduced with the following code.

```
decomposition <- function(f) {
  set.seed(1234)
  testxobs <- 1:100
  testyobs <- f(testxobs) + rnorm(100)
  decomposition.ols <- function(f) {
    x <- runif(50) * 100
    y <- f(x) + rnorm(50)
    lmest <- lm(y ~ x)
    predict(lmest, data.frame(x = testxobs))
  }
  a <- replicate(1000, decomposition.ols(f))
  a.lin <- c(mean((f(testxobs) - apply(a, 1, mean))^2), mean(apply(a,
    1, var)))

  decomposition.sm <- function(dof, f) {
    x <- runif(50) * 100
    y <- f(x) + rnorm(50)
    fit <- smooth.spline(x, y, df = dof)
    predict(fit, testxobs)$y
  }
  a.sm <- sapply(3:40, function(dof) {
    set.seed(1234)
    testxobs <- 1:100
    testyobs <- f(testxobs) + rnorm(100)
    a <- replicate(1000, decomposition.sm(dof, f))
    c(mean((f(testxobs) - apply(a, 1, mean))^2), mean(apply(a,
      1, var)))
  })
  decomp <- cbind(a.lin, a.sm)
  plot(2:40, decomp[1, ], type = "l", ylim = c(0, ceiling(max(decomp))),
    xlab = "flexibility", ylab = "", xaxt = "n", yaxt = "n",
    bty = "n", cex.lab = 3, col = "blue")
  axis(1, pos = 0, cex = 3, at = 0:40)
  axis(2, pos = 0, cex = 3)
  lines(2:40, decomp[2, ], col = "red")
  lines(2:40, 1 + apply(decomp, 2, sum), lwd = 3)
  abline(h = 1, lty = 2)
}

f1 <- function(x) 5 - 41.667 * (x/100)^3 + 51.875 * (x/100)^2 -
  9.125 * (x/100)
f2 <- function(x) 2 + 5 * ((x + 40)/100)^2
f3 <- function(x) -100 * (x/100)^3 + 150 * (x/100)^2 - 63 * (x/100) +
  12

layout(matrix(1:3, 1, 3))
decomposition(f1)
decomposition(f2)
decomposition(f3)
```

2.40

2.41